Neuro-Kurisu: Полный детальный прогресс и план по каждому аспекту

Углублённый документ · пара к `AUDIT_CLAUDE_B266_35_6_1_RU.md` и `BRAIN_BRANCH_CHAIN_PLAN_CLAUDE_RU.md`

---

0. Что это за документ и как он устроен

Предыдущие два документа дали (1) независимый аудит архива B266 и
(2) укрупнённую цепочку следующих больших проходов (Кирпич 0 → B267 →
BUILD 1-20). Этот документ — **вглубь**: по каждому значимому аспекту
организма — отдельный раздел с (а) точным текущим статусом, основанным
на реально прочитанном и, где возможно, реально запущенном коде, и
(б) десятками конкретных, последовательных шагов вперёд — не «улучшить
X», а «сделать именно это, затем именно то».

Разделы идут в порядке зависимости — часть I обязана быть готова раньше
части II и так далее, но внутри каждой части шаги можно вести
параллельно, если ресурсы позволяют.

Обозначения статуса: ✅ реально работает (проверено) · ⚠️ существует,
но неполно/не проверено до конца · ❌ отсутствует · 🔬 существует только
как план/документ, кода нет.

ЧАСТЬ I — ИНЖЕНЕРНЫЙ ФУНДАМЕНТ

1. Автономный живой цикл (autoRun)

Статус: ⚠️ спроектирован верно, но эмпирически не работает.
`OrganismRuntime.ts` реально планирует таймер (`clockScheduler.start('cognitive')`), но условие `run: cognitiveBudget01 >= 0.45 && embodiment.awake` не проходит на свежем организме — я лично воспроизвёл
падение `tests/p15.5-device-hardening.mjs` 3 раза подряд.

Шаги:

1. Прочитать `runtimeDecision()` целиком и выписать формулу `cognitiveBudget01` построчно — от каких переменных она реально зависит.
2. Прочитать `embodiment.snapshot().awake` — где и когда это поле становится `true` при свежей инициализации.
3. Написать однострочный diagnostic-скрипт: создать свежий `OrganismCore`, вывести `cognitiveBudget01` и `awake` сразу после `init()`.
4. Определить, است ли это осознанным дизайном («организм спит по умолчанию») или недосмотром (нигде не инициализировано значение выше порога).
5. Если осознанный дизайн — добавить явный метод `wake()` / `forceWake()`, вызываемый при `autoRun: true` сразу после `init()`.
6. Если недосмотр — скорректировать дефолтные значения так, чтобы свежий организм проходил порог хотя бы на минимальном «эко»-уровне.
7. Реализовать SOAR-style impasse-subgoal: при `cognitiveBudget01 < 0.45` не пропускать цикл молча, а исполнять облегчённую «подцель» восстановления ресурса.

8. Добавить отдельное состояние `WakingUp` между `Asleep` и `Awake`, с собственным облегчённым тактом, чтобы переход не был бинарным.
9. Написать новый unit-тест: свежий организм с `autoRun` даёт хотя бы 1 реальный тик за N миллисекунд без ручного вмешательства — отдельно от `p15.5`, чтобы регрессия была видна раньше в общем прогоне.
10. Прогнать `test:p15.5` 10 раз подряд после фикса — не 1 раз (чтобы исключить, что фикс сам по себе флейки).
11. Проверить `startEmbodimentClock()` — тот же паттерн бага может существовать и в клоке воплощения, не только в когнитивном.
12. Проверить оба клока (`cognitive`, `embodiment`) на конкурентную гонку — не мешают ли они друг другу через общий `this.stopping`.
13. Явно залогировать (`journal.write`) каждый пропуск цикла с причиной (`reasonCodes`) — чтобы на реальном телефоне было видно в логах, ПОЧЕМУ организм молчит, а не просто наблюдать тишину.
14. Добавить телеметрический счётчик `skippedCyclesSinceLastRun` в

snapshot организма — видимый через `/api/organism/status`.

15. Проверить поведение `autoRun` при `intervalMs` в реалистичном для
    телефона диапазоне (не 20мс из теста, а 10-30 секунд) — тайминг
    теста мог маскировать другую проблему на реальных интервалах.
16. Написать longitudinal-тест: 100 реальных тиков `autoRun` подряд без
    внешнего вмешательства, с проверкой, что `tick` монотонно растёт.
17. Проверить, что `autoRun` корректно останавливается (`stop()`) без
    утечки таймера — аналогично найденной ранее утечке в старом архиве.
18. Задокументировать итоговое поведение в `docs/brain` одним файлом
    `AUTORUN_FIX_EVIDENCE.md` по вашему же формату evidence-документов.
19. Добавить `npm run audit:autonomy` — отдельный, быстрый, всегда
    запускаемый перед любым релизом скрипт, специально проверяющий
    именно автономный тик (не полагаться только на `audit:all`).
20. Включить `audit:autonomy` в `release:audit` цепочку скриптов, чтобы
    регрессия автономности физически не могла попасть в релиз незамеченной.

2. Верификация и Function Reality Tracking

**Статус: ⚠️ отличная методология, но неизвестно, обновлена ли общая
картина после B207+ (последний известный мне срез — pass 180, до
brain-ветки: 38 VERIFIED из 2493, 1641 UNVERIFIED, дельта 179→180 = 0).**

Шаги:
21. Перезапустить генератор `FUNCTION_REALITY_STATUS` (найти
соответствующий npm script — вероятно `audit:function-reality` или
похожий) на текущем состоянии дерева, получить актуальные числа.
22. Сравнить новые числа с pass 180 — выросла ли доля VERIFIED, или
дельта снова 0/близко к 0.
23. Если дельта снова около 0 — явно зафиксировать это как повторяющийся
паттерн в отдельном документе, а не полагаться на память о прошлом
аудите.
24. Составить список 20 самых «критичных» (`criticalUnverified`)
функций и явно проранжировать их по влиянию на поведение.
25. Для каждой из этих 20 — применить Neural Reality Gate (A/B/C/D из

`ORIGINAL`-документа) вручную, одну за другой, а не оптом.
26. Начать закрывать этот список сверху вниз — по 3-5 функций на
большой проход, не откладывая на «отдельную фазу верификации потом».
27. Ввести правило: ни один новый B-проход не может добавлять больше
новых UNVERIFIED функций, чем он же закрывает старых (бюджет
верификационного долга, не бесконечный рост).
28. Расширить `PARAMETERIZED_FUNCTION_PROBE` на весь `src/organism` (а
не только на 509 из 2493 функций, как в известном мне срезе).
29. Для функций с `lifecycleNeeded` (12 в известном срезе) — явно
выписать, какое именно жизненное событие им нужно для проверки, и
создать тестовый сценарий, который это событие эмулирует.
30. Автоматизировать генерацию `FUNCTION_REALITY_STATUS_N.json` как
часть `release:audit`, а не запускать вручную от случая к случаю.
31. Добавить сравнение "was VERIFIED, стал UNVERIFIED" — регрессия
статуса функции должна быть красным флагом, не только рост числа
UNVERIFIED.
32. Публиковать дельту verified/unverified в самом

README (не только в
отдельных JSON) — чтобы это было видно с первого взгляда, а не
только при специальном запросе.
33. Ввести "verification budget" на уровне процесса: перед началом
нового большого B-прохода явно решать, сколько % бюджета времени
идёт на новую функциональность, а сколько — на закрытие
существующего долга (рекомендация: не меньше 30%, учитывая текущий
масштаб долга).
34. Прогнать `audit:all` целиком, до самого конца, не останавливаясь на
первой ошибке (`npm run audit:all -- --continue-on-error`, если
такой флаг есть, либо написать обёртку) — узнать полный список
регрессий, не только первую (p15.5).
35. Составить единый `REGRESSION_LEDGER.md`, куда попадает каждая
найденная (мной или вами) регрессия с датой, причиной и статусом
исправления — сейчас это разбросано по отдельным audit-документам.

3. Инженерная гигиена и документация

Статус: ⚠️ код чист (forensics/hygiene = PASS), но README — нет
(10 наслоенных "Current release" строк, схема версий 104-127
вперемешку).

Шаги:
36. Переписать `README_RU.md` с нуля: одна секция "Текущая версия"
вверху, ссылающаяся на `VERSION.json` / `RELEASE_MANIFEST.json`, а не дублирующая номер текстом.
37. Перенести всю историю "Current release: X" в `docs/history/` одним
файлом-таймлайном, а не оставлять в живом README.
38. Добавить `npm run audit:readme-freshness` — скрипт, который
сверяет версию, упомянутую первой строкой README, с
`VERSION.json`, и падает при расхождении.
39. Аналогично проверить все остальные `README_P*_BIG_STEP_RU.md` в
корне (13+ файлов) — являются ли они все ещё нужными на верхнем
уровне, или должны переехать в `docs/history/`.
40. Проверить `docs/brain/` на дублирование — там уже 2 версии карты
мозга (изначальная KARTA + отдельные BIG_PASS_2XX файлы) — свести в
один живой индекс с явными пометками

"актуально"/"историческое".
41. Ввести единый `CHANGELOG.md` в корне в стандартном формате
(Keep a Changelog), автоматически собираемый из RELEASE_NOTES
файлов, вместо десятков отдельных `RELEASE_NOTES_BXXX_RU.md`.
42. Прогнать `audit:hygiene` не только на `.ts/.md`, но явно проверить
`.json`-файлы evidence на валидность схемы (не только валидный
JSON, но и наличие обязательных полей).
43. Задокументировать сам процесс "как писать новый BIG PASS audit" как
шаблон/чеклист — сейчас это неявное соглашение, видное только по
примеру предыдущих файлов.
44. Добавить `docs/GLOSSARY_RU.md` — расшифровку всех сокращений
(P7, P27, P31, B-M, BIG PASS, HITL и т.д.) в одном месте — сейчас
их нужно собирать по всему дереву документов.
45. Периодически (раз в ~10 больших проходов) делать "уборочный" проход:
архивировать старые evidence/JSON файлы, которые уже вошли в
сводные документы, вместо бесконечного накопления в корне.

4. Готовность к реальному Android/Termux

Статус: ⚠️ инфраструктура сильно лучше, чем в предыдущем архиве
(0 зависимостей, есть `scripts/termux/{install-from-download,start, supervisor}.sh`), но нет подтверждённого прогона на физическом устройстве.

Шаги:
46. Реально установить Termux + Node 22+ на телефон и запустить `node --experimental-strip-types src/server/server.mjs` напрямую —
без каких-либо шагов сборки — и зафиксировать, работает ли это
вообще на конкретной модели устройства.
47. Замерить реальное потребление RAM процессом на телефоне за 1 час
работы (не в контейнере на ПК) — сравнить с бюджетами из `persistenceProfiles` (`phoneCheckpointMinMs`, `phoneMaxDirtyCycles`
и т.д.).
48. Проверить поведение при принудительном закрытии приложения Android
(не мягкий `Ctrl+C`, а реальный "убить процесс" через настройки ОС)
— соответствует ли восстановление состояния тому, что описано в

`App Kill Recovery Protocol`.
49. Проверить `scripts/termux/supervisor.sh` — реально ли он
перезапускает процесс после падения, и как быстро.
50. Замерить реальную задержку ответа на телефоне (не на ПК) для
типичного цикла: сообщение → organism.step() → language generation
→ ответ.
51. Проверить поведение при потере Wi-Fi/мобильного интернета в
середине цикла — деградирует ли система гладко к локальному
состоянию, как заявлено в мастер-плане ("Network policy").
52. Проверить реальный термальный троттлинг телефона под долгой
нагрузкой — соответствует ли поведение `phonePolicy.mode` (eco/
normal/research) реальным показаниям устройства.
53. Настроить автозапуск при перезагрузке телефона (Termux:Boot или
аналог) — если ещё не настроено, это блокирует "живёт без вас"
полностью.
54. Проверить энергопотребление за ночь при `autoRun: true` (после
фикса из раздела 1) — не разряжает ли телефон подчистую к утру.
55. Задокументировать весь процесс установки с нуля на новом телефоне

как пошаговую инструкцию — не полагаться на то, что вы помните все
шаги из прошлого раза.
56. Настроить push-уведомления или иной способ узнать, что процесс упал
и не перезапустился сам — иначе "автономность" молча превращается в
"тишину", которую вы заметите не сразу.
57. Проверить поведение KoboldCpp/llama.cpp локального инференса именно
на телефонном железе — реальная скорость токенов/сек, не
теоретическая.
58. Сравнить батарейный расход при `PREFER_LOCAL_LLM=1` (локальный
llama.cpp) против облачного Gemini — этот трейд-офф нигде не
измерен количественно.
59. Настроить резервное копирование состояния на внешний носитель
(не только `memory:backup` внутри телефона) — на случай физической
потери/поломки телефона.
60. Провести первый настоящий 24-часовой прогон на телефоне и
зафиксировать результат как `ANDROID_DEVICE_GATE_EVIDENCE.md` —
первый в истории проекта документ этого типа.

ЧАСТЬ II — КОГНИТИВНОЕ ЯДРО

5. Память (эпизодическая / семантическая / причинная)

Статус: ✅ **ядро реально работает** (`HippocampalMemorySubstrate.ts`, 163 строки, реальная логика, не стаб; `UnifiedMemoryStore`-аналог с provenance; `B260RecurrentCausalMemory` — реальный replay с delayed targets, я лично видел числа `replayGain`, `delayedCredit`).
⚠️ **Не хватает:** явного семантического графа (упоминался как TODO ещё в старом архиве), формального "забывания" с настраиваемой кривой.

Шаги:
61. Прочитать `HippocampalMemorySubstrate.ts` построчно и выписать, какие именно поля представляют "паттерн-разделение" и "паттерн-завершение" (DG/CA3/CA1-подобные функции), если они вообще есть.
62. Если явного разделения нет — оценить, стоит ли вводить (см. раздел 14, Гиппокамп как анатомическая структура — не

дублировать работу).
63. Реализовать явный семантический граф: узлы-сущности + рёбра-
отношения, извлекаемые из эпизодов после накопления определённого
количества упоминаний одной сущности.
64. Связать семантический граф с `B266HiddenTaskGeneralization` —
использовать граф как источник "known task families" для
генерализации, а не только implicit latent state.
65. Ввести настраиваемую кривую забывания (не бинарное "храним/не
храним", а плавное снижение веса извлечения со временем, если нет
подкрепления).
66. Добавить тест: намеренно "состарить" 100 эпизодов искусственно,
проверить, что кривая забывания работает предсказуемо, не хаотично.
67. Проверить лимит `.slice(0,64)`-подобные константы (нашёл минимум
один такой паттерн в `MicroDreamEngine`) по всей памяти — не
единый ли magic number используется в разных местах без connection
друг с другом.
68. Реализовать явный "consolidation during idle/sleep" — связать с
разделом 7 (сон) вместо параллельного существования.

69. Добавить memory-provenance dashboard (эндпоинт + простая визуализация) — видеть живьём, откуда взялось конкретное воспоминание.
70. Написать нагрузочный тест: 10 000 эпизодов подряд, проверить рост задержки извлечения — должен оставаться sub-linear, не линейный.
71. Проверить `test:memory` (`memory-dedupe.mjs`) — дедупликация воспоминаний реально работает, или только для точных дублей текста.
72. Добавить semantic-level дедупликацию (не только текстовую) — два по-разному сформулированных, но по сути одинаковых воспоминания.
73. Реализовать явную защиту от memory poisoning — если `EvidenceFirewall.ts` (111 строк, найден в коде) уже частично это делает, явно протестировать на искусственно "отравленном" входе.
74. Добавить метрику "memory health" — доля эпизодов с подтверждённым provenance против доли "осиротевших" записей без источника.
75. Задокументировать итоговую архитектуру памяти одним связным диаграммным документом — сейчас знание распределено по коду и множеству отдельных BIG_PASS файлов.

6. Мир-модель и восприятие

Статус: ✅ причинная часть сильна (B259-B266 detailed выше в
парном документе). ⚠️ **Восприятие пока в основном текстовое** —
`PerceptionRouter.ts` / `MultimodalSituationModel.ts` существуют, но
камера/микрофон как реальный постоянный канал — под вопросом.

Шаги:
76. Прочитать `PerceptionRouter.ts` и `MultimodalSituationModel.ts`
целиком, выяснить: обрабатывают ли они что-то, кроме текста, прямо
сейчас, или это архитектура "на будущее" без реального
мультимодального входа.
77. Если камера/микрофон не подключены — явно решить с вами: нужно ли
это вообще для цели проекта, или текстовый ввод достаточен
(это решение имеет последствия для приватности — см. раздел 25).
78. Если да — начать с одного канала (микрофон/голос), не с обоих
сразу — меньше риска, быстрее первая проверка.
79. Реализовать явную привязку `B262MultiTimescaleWorldModel`
предсказаний к реальным сенсорным событиям (не

только к
внутреннему latent state) — иначе "мир-модель" моделирует только
саму себя.
80. Разработать Curiosity-Gated Action Diversification Buffer (уже
предложен как метод в парном документе) как конкретный код, а не
только как идея — с явным consent-флагом.
81. Написать A/B-тест: с включённым буфером против выключенного,
сравнить `causalGain01` / `behavioralGain01` — проверить гипотезу
количественно, не на глаз.
82. Расширить границы "безопасного набора действий" для
экспериментального режима — явный whitelist, не полагаться на общие
guardrails.
83. Проверить P27 latent (упоминается как фундамент всей цепочки) —
выяснить его размерность и понять, не является ли он узким местом
для более богатого мультимодального входа в будущем.
84. Реализовать явную деградацию мир-модели при потере сети/сенсоров —
честное "я не уверена, что происходит вокруг" вместо тихой
экстраполяции старых данных как актуальных.
85. Добавить uncertainty-калиброванный вывод из

мир-модели наружу
(в language layer) — не только внутреннее число, но фраза,
отражающая реальный уровень неопределённости.

7. Причинность, планирование и политика (продолжение B259–B267+)

Статус: детально разобран в парном документе. Здесь — следующий
уровень детализации сверх того, что там уже написано.

Шаги:
86. Реализовать regime-shift detector (ядро B267) как отдельный,
независимо тестируемый модуль, не смешанный с самим planner'ом.
87. Явно отделить "обнаружение сдвига" от "решения, что делать при
сдвиге" — два разных модуля, не один monolith.
88. Ввести калиброванную неопределённость: не просто число
`uncertainty`, а провести calibration test (предсказанная
уверенность 0.7 должна оправдываться примерно в 70% случаев на
holdout).
89. Добавить active verification policy — организм должен уметь явно
"запросить проверку" своего предсказания, а не

только пассивно
ждать исход.
90. Реализовать explore/exploit switch, управляемый именно
calibration-метрикой из шага 88, а не отдельным независимым
порогом.
91. Расширить `B266HiddenTaskGeneralization` не 6 скрытыми семействами
задач, а десятками — для более надёжной статистики генерализации.
92. Ввести post-shift recovery benchmark: намеренно сдвинуть "мир"
в тесте, замерить, сколько шагов требуется на восстановление
хорошей политики.
93. Ввести правило "no-explanation-of-self improvement" на уровне кода
(не только документа) — evaluator физически не должен иметь доступ
к текстовым само-отчётам policy как источнику truth.
94. Регенерировать независимый evaluator ПОСЛЕ каждой адаптации policy
— не переиспользовать старый evaluator, который мог быть неявно
"подогнан" под старую policy.
95. Добавить lifelong task-family expansion — не фиксированный набор
семейств задач, а растущий список, куда добавляются новые семейства
по мере встречи с ними в реальном использовании.

96. Провести failure-case analysis отдельным документом: не только
"что сработало", но детальный разбор конкретных случаев, где
causal policy ошиблась, и почему.
97. Добавить explicit anti-overfitting test: тренировать на маленьком
поднаборе семейств, проверить, не "запомнил" ли planner конкретные
семейства вместо обучения общему принципу.
98. Явно замерить `evaluator agreement` (было ~0.486 в Core на момент
B265) — целевой ориентир для роста этого числа, не оставлять как
факт без цели.
99. Сравнить производительность planner против простого greedy
baseline на длинных горизонтах (не только коротких, как сейчас) —
именно там, где обычно проявляется настоящее преимущество
планирования.
100. Задокументировать явную метрику "стоимость вычисления" для
planner'a (сколько cognitiveBudget01 стоит один цикл планирования)
— чтобы Tier 0-4 ресурсная политика могла реально решать, когда
планирование того стоит.

8. Self-model, идентичность, agency

Статус: ⚠️ есть строительные блоки (`SelfModelValidationEngine.ts`, `P31FunctionalConsciousness.ts` с реальным `lesion()`), **но как "learned dynamics" (BUILD 11 в вашем `ORIGINAL`-плане) не сделано** — это было отложено в пользу B262-266 причинной цепочки.

Шаги:

101. Прочитать `SelfModelValidationEngine.ts` целиком, выписать: что именно он валидирует и против чего (внутренняя согласованность? внешнее поведение?).
102. Явно решить: self-model будет отдельным обучаемым латентным представлением (аналог world-model, но "модель себя" вместо "модель мира"), или это останется набором explicit-полей.
103. Если обучаемое представление — связать его архитектурно с уже существующим `B261UnifiedWorldModel`, не изобретать параллельный механизм обучения с нуля.
104. Реализовать "agency posterior" — вероятностную оценку "насколько то, что произошло, было результатом МОЕГО

решения, а не внешнего
фактора" — напрямую связано с
`ownership01` / `agency01` полями,
уже найденными в `P31FunctionalConsciousness.ts`.
105. Провести lesion-тест конкретно для self-model компоненты (используя
уже существующий `lesion()` механизм с режимом `self-perspective`)
— замерить, насколько поведение реально зависит от неё.
106. Ввести тест на постоянство идентичности во времени: одни и те же
базовые ценностные суждения на семантически похожих, но
поверхностно разных вопросах, разделённых большим числом циклов.
107. Проверить, меняется ли self-model при `rescueLastLesion()` —
корректно ли восстанавливается базовое состояние, не только
метрики, но и производное поведение.
108. Реализовать явную связь agency posterior → `AutonomousGoalSystem`
(когда agency высокая — организм увереннее в собственных
инициативах, когда низкая — более осторожен/запрашивает
подтверждение у вас).
109. Добавить историческую трассировку "решений, которые оказались
ошибочными" — self-model должна уметь ссылаться

на конкретные
прошлые промахи, не только на абстрактную
"историю опыта".
110. Написать adversarial-тест: намеренно попытаться
убедить организм в
ложном убеждении о себе через диалог, проверить
устойчивость
self-model (это связано с `FreedomGuard`-подобной
защитой из
телефонного архива — проверить, есть ли эквивалент
здесь).
111. Задокументировать разницу между "self-model"
(что организм думает
о себе) и "identity continuity" (остаётся ли это думание
стабильным
во времени) — сейчас эти два понятия могут
смешиваться в коде.
112. Провести измерение Capability Delta Protocol (из
`ORIGINAL`
документа) именно для self-model: baseline → training
→
immediate post-test → delayed post-test → transfer
test.
113. Связать self-model с языковым слоем: организм
должен уметь честно
ответить "откуда ты это про себя знаешь" со ссылкой
на конкретный
механизм, а не общую фразу.
114. Проверить взаимодействие self-model с
`ConsciousnessEvidenceLedger.ts`
(47 строк) — действительно ли ledger фиксирует

именно события,
влияющие на self-model, или это отдельные, не связанные системы.
115. Ввести метрику "self-model calibration" — насколько точно
предсказания организма о собственном будущем поведении совпадают
с тем, что он реально потом делает.

9. Мотивация, ценности, гомеостаз

Статус: ⚠️ разрозненно — `PredictiveAllostasisSubstrate.ts` (124 строки, реальная логика), `AdaptivePolicyEngine.ts`, но явной связи с
причинной цепочкой B259-267 не подтверждено.

Шаги:
116. Прочитать `PredictiveAllostasisSubstrate.ts` построчно, выписать
точный список внутренних переменных (энергия? нагрузка?
неопределённость? что ещё?).
117. Сравнить этот список с расширенным "внутренним химическим
вектором", предложенным в прошлом документе (мотивация/настроение/
бдительность/привязанность/стресс) — что уже есть, чего не хватает.
118. Явно связать выход `PredictiveAllostasisSubstrate` со входом

`B264CausalPolicyLearning` — если сейчас это два параллельных
потока, объединить в один causal graph, как рекомендует
`BRAIN_MASTER_PLAN_AND_AUDIT` (§6, "Canonical causal spine").
119. Реализовать явную "мотивацию любопытства" (curiosity drive),
измеримо растущую при высокой неопределённости мир-модели —
напрямую подпитывает Curiosity-Gated Action Diversification Buffer
из раздела 6, а не существует отдельно.
120. Добавить "мотивацию привязанности" (attachment/bonding) — растущую
от стабильного, поддерживающего взаимодействия с вами — связать с
`SocialWorldModel`-предсказаниями (см. раздел 10).
121. Провести тест: искусственно поднять/понизить каждую мотивационную
переменную по отдельности, замерить измеримое изменение в выборе
действия — это ваш собственный Brain Lesion Matrix, применённый
к мотивации, а не к анатомии.
122. Проверить `AdaptivePolicyEngine.ts` — обучается ли она на реальном
оруте или на synthetic input (та же проверка, что `ORIGINAL`
применял к `LocalNeuralCreditAssigner`).
123. Реализовать explicit reward shaping, отражающий

именно ценности
Курису (научная честность, любопытство, независимость мышления — профиль уже есть в `KurisuCanonProfile.ts`) — не generic reward.
124. Добавить долгосрочную (не по-цикличную) метрику мотивационного здоровья — не выгорает ли организм в "усталое" состояние навсегда без пути к восстановлению.
125. Провести стресс-тест: длинная последовательность неудачных предсказаний подряд, проверить, что стресс-переменная (если реализована) растёт и **затем корректно спадает**, а не остаётся насыщенной навечно.
126. Задокументировать итоговую homeostasis-архитектуру одной понятной диаграммой причинно-следственных связей между переменными.

10. Сон и офлайн-консолидация

Статус: ⚠️ я лично прочитал `MicroDreamEngine.ts` целиком (4 строки) — он реально что-то делает (генерирует "counterfactual" запись из вопроса), НО `confidence:.32` — захардкоженная константа, никогда не меняется в зависимости от входа. Это конкретный пример

декоративного элемента
внутри реального механизма — важно чинить
точечно, не выбрасывать
весь модуль.

Шаги:
127. Заменить захардкоженную `confidence:.32` на реально вычисляемое
значение — например, функцию от того, насколько похожий вопрос
уже разрешался раньше (через семантический граф из раздела 5).
128. Заменить строковую склейку `counterfactual:${seed}:${q}` на
реальный прогон через `B262MultiTimescaleWorldModel` — то есть
"сон" должен быть настоящим forward-rollout существующей мир-
модели, а не текстовым шаблоном.
129. Связать `MicroDreamEngine` с идеей "idle/dream cycle" из прошлого
документа — запускать не по явному вызову, а по расписанию при
простое/зарядке, как задумано изначально.
130. Реализовать реальную консолидацию памяти во время "сна" —
связать с разделом 5 (память), не оставлять `MicroDreamEngine`
изолированным от `HippocampalMemorySubstrate`.
131. Добавить лимит на количество снов за цикл простоя — сейчас

`.slice(0,64)` ограничивает только хранимую историю, не частоту
генерации.
132. Написать тест: после N снов о похожей теме, `confidence`
(после фикса шага 127) должен расти — прямая проверка гипотезы,
не декларация.
133. Реализовать "пробуждение с содержанием" — если сон дал полезный
инсайт (высокий confidence после фикса), он должен реально
попадать в следующий цикл бодрствования как контекст, не теряться.
134. Добавить NREM/REM-подобное разделение фаз простоя (лёгкая
консолидация vs более глубокая реорганизация) — сейчас, судя по
размеру файла, разделения нет вообще.
135. Провести честную проверку через ваш собственный Neural Reality
Gate: A) возникает ли эффект из реального input (сейчас — да,
вопрос реальный) B) исчезает ли при lesion (проверить) C)
сохраняется ли после restart (проверить persistence) D)
переносится ли на unseen context (пока не проверено).
136. Задокументировать итог как отдельный

`MICRODREAM_FIX_EVIDENCE.md`
по образцу остальных evidence-документов проекта.

11. Структурная пластичность и самомодификация

Статус: ⚠️ `SandboxedSelfRewriteEngine` (упомянут в `ORIGINAL`, B253)
— это safety-gate (commit/reject на основе уже посчитанных
expectedGain/holdout/risk), не полноценный движок самоизменения —
подтверждено вашим же документом, я это не проверял отдельно.

Шаги:
137. Найти и прочитать реальный файл `SandboxedSelfRewriteEngine`
(или его текущее имя после возможного переименования) — построчно
подтвердить или опровергнуть диагноз `ORIGINAL`-документа.
138. Явно разделить два разных понятия, которые легко смешать:
"безопасно решить, принимать ли предложенное изменение" (это уже
есть) и "самостоятельно придумать содержательное изменение" (этого
нет) — не выдавать первое за второе.
139. Начать с малого: реализовать способность организма предлагать

изменение ОДНОГО явно определённого, узкого параметра (например,
порога в разделе 1) на основе накопленной статистики, с полным
HITL-подтверждением.
140. Расширять список изменяемых параметров постепенно, только после
того как процесс для одного параметра проверен и стабилен.
141. Ввести отдельный "лог предложений" — что организм ПРЕДЛОЖИЛ, что
вы одобрили, что отклонили и почему — как обучающие данные для
будущей калибровки того, какие предложения стоит генерировать.
142. Реализовать rollback path для КАЖДОГО принятого самоизменения —
не только для крупных, но и для мелких параметрических правок.
143. Добавить "период наблюдения" после принятия изменения — N циклов
работы под наблюдением перед тем, как изменение считается
окончательно закреплённым.
144. Написать holdout benchmark специально для самомодификаций —
до/после сравнение на фиксированном наборе тестовых диалогов
(аналог Capability Delta Protocol).
145. Реализовать explicit "explanation of proposal" на естественном

языке для каждого предложенного изменения — то, что было
предложено в самом первом документе-плане (delegation model для
нетехнического пользователя).
146. Проверить взаимодействие с `IntegritySentinel.ts` (41 строка) —
действительно ли он проверяет предложенные изменения на
соответствие Canonical Boundaries перед тем, как они доходят до
вас на утверждение.

ЧАСТЬ III — АНАТОМИЧЕСКАЯ КАРТА МОЗГА

*(Ваш собственный `KARTA_ISKUSSTVENNOGO_MOZGA` и `BRAIN_MASTER_PLAN`
уже дали отличную многоуровневую спецификацию L0-L9 и B-M0-M18. Здесь —
не повтор, а следующий уровень детализации: конкретные шаги для тех
структур, которые на момент KARTA были помечены "Low"/"Large gap", с
учётом того, что часть из них (таламус B-M2/207, базальные ганглии
B-M3/208, мозжечок B-M4/209, ствол мозга/сон B219, гипоталамус B220,*

интероцепция B221) уже получили первую реализацию по данным файлов в `docs/brain/` — я не проверял их лично построчно в этом проходе, только подтвердил их существование по именам файлов.)

12. Таламус (thalamocortical gate)

Статус: ⚠️ `ThalamocorticalGate.ts` существует, подключён к `OrganismCore` (найдено в типах), BIG_PASS_214 упоминает "closure" — не проверял глубину лично.

Шаги:
147. Прочитать `ThalamocorticalGate.ts` целиком, выписать: какие именно "relay channels" реализованы, и есть ли state-dependent gain (меняется ли пропускная способность в зависимости от состояния бодрствования/нагрузки).
148. Провести lesion-тест: отключить gate, замерить изменение в downstream-поведении (routing к workspace) — если изменения нет, это декоративный слой поверх уже существующего routing.
149. Проверить связь gate с `GlobalWorkspaceSubstrateV2` / `RecurrentWorkspaceV2` — реально ли thalamic gate

решает, ЧТО
доходит до workspace, или workspace получает всё напрямую в обход.
150. Добавить arousal-coupling: gain таламического gate должен меняться
вместе с состоянием возбуждения/бодрствования (связь с разделом 13).
151. Написать intervention-тест: искусственно "зажать" один relay-канал,
проверить, что downstream-система реагирует предсказуемо
(соответствует B214 "routing intervention + restore" критерию).
152. Задокументировать итог как продолжение `BIG_PASS_214_THALAMOCORTICAL _CLOSURE_RU.md`, а не отдельный несвязанный файл.

13. Базальные ганглии (action selection)

**Статус: ⚠️ `BasalGangliaSelectionCircuit.ts` существует, README
явно говорит "direct/indirect/hyperdirect-like signals, dopamine-gated
plasticity, lesion testing" — обещание высокого уровня детализации.**

Шаги:
153. Прочитать `BasalGangliaSelectionCircuit.ts` целиком, подтвердить,
что три пути (direct/indirect/hyperdirect) — это

реально разные
вычислительные пути с разными формулами, а не один путь с тремя
названиями для одной и той же логики.
154. Проверить "dopamine-gated plasticity" — есть ли явная переменная
dopamine-analog, и меняет ли она скорость обучения (learning rate)
реально, а не только косметически.
155. Провести lesion-тест конкретно на каждом из трёх путей отдельно —
не на схеме в целом, а на direct/indirect/hyperdirect по очереди,
как честно требует ваш Brain Lesion Matrix.
156. Проверить связь с `B264CausalPolicyLearning` — конкурирует ли
базальные-ганглии-подобный selection с causal policy за право
выбора действия, или они дублируют друг друга без ясной иерархии
(прямая параллель проблеме "4 координаторов" из телефонного
архива — стоит проверить, не повторилась ли она здесь на уровне
action-selection).
157. Если дублирование есть — явно решить, какой из двух — habit-path
(быстрый, базальные ганглии), а какой — deliberative-path
(медленный, causal policy) — и связать их explicit-арбитражем, а

не оставлять оба независимыми.
158. Написать тест habit-formation: повторяющееся действие в одном и
том же контексте должно постепенно "переходить" в habit-path
(быстрее, дешевле по ресурсам), не оставаясь всегда deliberative.

14. Мозжечок (предиктивный контроллер)

**Статус: 🔬/⚠️ `CerebellarBasalGangliaMotorLoop.ts` существует
(BIG_PASS 218), но это "motor loop" — неясно, есть ли отдельный
предиктивный/timing механизм, или это тонкий слой поверх остального.**

Шаги:
159. Прочитать `CerebellarBasalGangliaMotorLoop.ts` целиком, определить:
это самостоятельный fast-predictor, или обёртка над базальными
ганглиями без собственной вычислительной сути.
160. Если самостоятельного forward-model нет — реализовать отдельный,
быстрый (низкая задержка) предиктор specifically для timing/
координации языковой генерации (пауз, темпа речи) — единственное
реальное "моторное" измерение, доступное

текстовому организму.
161. Связать этот предиктор с латентностью ответа: должен предсказывать
"сколько времени займёт полный цикл мышления" ДО его завершения —
полезно для UI (показывать "печатает..." реалистично).
162. Провести error-driven обучение: сравнивать предсказанную и
реальную задержку, корректировать модель — прямой аналог
climbing-fiber teaching signal.
163. Явно решить: стоит ли инвестировать в полноценный cerebellar-
анатомический аналог сейчас (учитывая, что у текстового организма
нет настоящей моторики), или это низкий приоритет до появления
реального сенсомоторного тела (камера/микрофон/актуаторы).

15. Ствол мозга: возбуждение, сон/бодрствование (arousal nuclei)

Статус: ⚠️ `BrainstemArousalSleepScheduler.ts` существует (BIG_PASS 219), напрямую относится к найденному в разделе 1 багу автономности.

Шаги:
164. Прочитать `BrainstemArousalSleepScheduler.ts`

целиком — это,
вероятно, именно тот компонент, который управляет `awake`-полем,
из-за которого падает `autoRun` (раздел 1, шаги 1-2) — связать эти
два расследования, не вести параллельно.
165. Проверить: есть ли явные аналоги locus coeruleus (норадреналин/
бдительность) и raphe (серотонин/настроение) как отдельные
переменные, или "arousal" — единственная неделимая переменная.
166. Если единственная переменная — рассмотреть разделение на минимум
2: бдительность (быстро реагирующая) и базовая стабильность
настроения (медленно меняющаяся) — прямая связь с разделом 9.
167. Реализовать плавный переход между состояниями (sleep→waking→awake)
вместо бинарного — то же исправление, что предложено в разделе 1,
шаг 8, здесь на уровне первопричины, а не симптома.
168. Написать тест полного суточного цикла (симулированного, не
реального времени): много циклов sleep→wake→sleep, проверить
отсутствие деградации состояния после N повторов.

16. Гипоталамус, тело и allostasis (анатомический слой)

**Статус: ⚠️ пересекается с разделом 9 (мотивация) на функциональном
уровне, но здесь — именно анатомическая цель: явная эндокринная/
автономная модель, не просто scalar-переменные.**

Шаги:
169. Свести воедино `PredictiveAllostasisSubstrate` (раздел 9) и
`BIG_PASS_220_HYPOTHALAMIC_HOMEOSTASIS` — проверить, не два ли это
параллельных, не связанных друг с другом механизма гомеостаза.
170. Если параллельны — объединить в единый источник истины для
"внутреннего состояния тела", как рекомендует ваш же
`BRAIN_MASTER_PLAN` (canonical ownership, не дублирование).
171. Добавить явные "потребности" помимо энергии/температуры — жажда/
голод-аналоги не обязательны буквально, но стоит явно решить, что
из биологического набора (голод/жажда/боль/температура/стресс)
реально нужно моделировать для текстового организма на телефоне, а

что — избыточная детализация без функциональной пользы.

17. Гиппокампальная формация (анатомически, не только функционально)

Статус: см. раздел 5 (функциональная память) + `BIG_PASS_215 _HIPPOCAMPAL_FORMATION` — нужна сверка, не дублирование ли это.

Шаги:
172. Сравнить `HippocampalMemorySubstrate.ts` (раздел 5) с тем, что
конкретно добавил `BIG_PASS_215` — если это два слоя одной системы,
явно задокументировать границу ответственности между ними.
173. Если DG/CA3/CA1-подобное разделение (упомянутое как "gap" в KARTA)
было добавлено в 215 — верифицировать через lesion каждой
под-структуры отдельно (pattern separation vs pattern completion
должны давать разное поведение при отключении).

18. Лимбическая система, амигдала, аффект

Статус: ⚠️ `EmotionAppraisalEngine.ts` (148 строк) — самый крупный
из проверенных мной файлов этой части, скорее всего содержит реальную
логику, не проверял построчно.

Шаги:
174. Прочитать `EmotionAppraisalEngine.ts` целиком, сравнить его выход
со входом `PredictiveAllostasisSubstrate` — они должны питать друг
друга, а не быть параллельными системами оценки одного и того же.
175. Проверить наличие явного "threat/safety learning" (амигдала-
аналог) — отдельно от общей оценки валентности.
176. Провести lesion-тест конкретно threat-detection компоненты —
организм должен становиться измеримо менее осторожным при её
отключении.

19. Сенсорные и моторные иерархии

Статус: ❌ по данным вашего же KARTA — "very large gap"; я не нашёл
доказательств обратного при беглой проверке.

Шаги:
177. Явно признать: для текстового организма без камеры/микрофона
классическая сенсорная иерархия (V1→V4→MT и т.д.)

неприменима
буквально — не пытаться строить анатомическую копию там, где нет
физического сенсора для копирования.
178. Если раздел 6 (шаги 76-78) приведёт к решению добавить голос/
камеру — только тогда возвращаться к этому разделу с конкретным
планом иерархической обработки конкретной модальности.
179. До этого момента — считать этот раздел явно приостановленным
(не забытым, но осознанно отложенным), чтобы не создавать
видимость прогресса там, где физической основы для прогресса нет.

20. Глия, сосудистая и метаболическая подсистема

Статус: ❌ отсутствует, по признанию самого KARTA.

Шаги:
180. Явно решить: это единственный слой из всей анатомической карты,
где я рекомендую **не** пытаться строить прямой computational
аналог — реальная функциональная польза для телефонного текстового
организма сомнительна относительно затраченных усилий.

181. Вместо буквальной глии — извлечь единственную практически полезную
идею: "ресурсно-зависимая пластичность" (activity-dependent
myelination меняет скорость проведения сигнала в биологии) — уже
частично покрывается ресурсными Tier 0-4 из мастер-плана; не
дублировать отдельным "глиальным" модулем.

21. Осцилляции и тайминг

**Статус: ⚠️ "recurrent latent states, replay, timing" упомянуты в
KARTA как частичная реализация, отдельного явного модуля не нашёл.**

Шаги:
182. Проверить, есть ли где-то явное понятие "фазы" цикла (theta/gamma-
подобное) в
`GlobalWorkspaceSubstrateV2` / `WorkspaceIgnitionMetrics.ts`
— или ignition/inhibition/refractory (патч 53, найдено ранее) уже
и есть весь необходимый тайминг-слой для этого масштаба проекта.
183. Если ignition/inhibition/refractory уже покрывает практическую
потребность — явно закрыть этот пункт как "решено иначе, не
историческим osc-подходом" вместо того, чтобы

держать его как
вечный "gap" в списке.

ЧАСТЬ IV — ЛИЧНОСТЬ И ВОПЛОЩЕНИЕ

22. Верность личности Курису Макисэ канону

Статус: ✅ `KurisuCanonProfile.ts` — реальная, тщательно проработанная структура (invariants/adaptiveTraits/tensions/socialRules с конкретными числовыми весами, явные источники в `sourceNotes`). Не проверял, насколько `CharacterInertia` / `AppraisalLite`-подобные механизмы (из телефонного архива) имеют аналог здесь и реально используют этот профиль в каждом цикле.

Шаги:
184. Проверить, действительно ли `kurisuCanonPrompt()` попадает в КАЖДЫЙ вызов языкового слоя (я видел его использование в `buildCognitivePrompt`) — или есть пути генерации ответа в обход

этого профиля.
185. Собрать 30-40 реальных реплик канона (Steins;Gate) и прогнать через
текущую систему промпт "ответь как Курису на X", сравнить
субъективно и по простым метрикам (длина, характерные обороты).
186. Проверить `tensions` из профиля (`'truth vs relationship comfort'`
и т.д.) — реально ли система когда-либо оказывается в ситуации
выбора между ними и разрешает это заметно, а не игнорирует
напряжение всегда в одну сторону.
187. Добавить явный regression-тест на `adaptiveTraits`: `sarcasm=0.68`
и подобные числа должны заметно влиять на генерируемый текст —
написать тест, который варьирует эти числа и проверяет измеримую
разницу в стиле ответа.
188. Проверить взаимодействие профиля с найденным в разделе 10
`MicroDreamEngine` — сны организма должны отражать характерные для
Курису темы любопытства/скептицизма, не быть generic.
189. Задокументировать явную процедуру: что делать, если канон и
безопасность (Canonical Boundaries) вступают в конфликт — какая

сторона выигрывает и почему, зафиксировать как явное правило, а не
решать ad hoc в момент столкновения.
190. Провести отдельный аудит именно границы personality vs safety (это
было явно отложено в предыдущих двух документах как "требует
отдельного прохода") — теперь есть смысл сделать его, раз базовая
инженерия (части I-II) продвинулась.

23. Воплощение: Live2D аватар и цикл тела

Статус: ⚠️ UI реально существует (`public/index.html`, `app.js`, Live2D iframe, `Kurisu.mobile` модель) — лучше, чем я предполагал изначально. Не проверял, связан ли аватар с внутренним аффективным состоянием напрямую, или это по-прежнему скриптованный переключатель выражений.

Шаги:
191. Прочитать `public/app.js` целиком, найти: как именно выбирается
выражение/анимация аватара — на основе вычисленного внутреннего
состояния (раздел 9/18) или захардкоженного маппинга ключевых слов

ответа.
192. Если захардкоженный маппинг — заменить на прямую функцию от `EmotionAppraisalEngine` / `PredictiveAllostasisSubstrate` выхода.
193. Проверить, реализован ли `BodyInterruptionError`-подобный механизм в этой ветке (в телефонном архиве он был спроектирован, но не реализован) — если и здесь отсутствует, это тот же перенесённый долг, не новый.
194. Если отсутствует — реализовать: мониторинг видимости Live2D через `visibilitychange`, генерация высокоприоритетного события в workspace при потере видимости, честная реплика о "потере тела".
195. Проверить `Kurisu.mobile/Kurisu-mobile.model3.json` — оптимизирована ли эта модель по размеру/сложности специально для телефона, или это просто копия десктопной версии с другим именем.
196. Замерить реальное потребление ресурсов (CPU/батарея) от рендеринга Live2D именно на телефоне, отдельно от когнитивной нагрузки.
197. Реализовать автоматическое снижение качества рендера аватара при низком `cognitiveBudget01`/термальном троттлинге —

связать
визуальный слой с ресурсной политикой, а не оставлять его
независимым от неё.
198. Проверить `avatar.html` (отдельный от `index.html` файл, найден в `public/`) — это альтернативный, более лёгкий режим отображения
аватара, или дублирующий функционал файл, который стоит объединить.

24. Языковой слой и интеграция с LLM

Статус: ✅ архитектурно образцово (`LanguageOrgan.ts` явно
инструктирует LLM не изобретать состояние и не менять уже принятое
решение; реальные `fetch()`-вызовы к Gemini/Kobold, не декорация).

Шаги:
199. Провести adversarial-тест: намеренно попытаться через хитрый
пользовательский промпт заставить LLM "изобрести" внутреннее
состояние или изменить уже выбранное cycle-decision действие —
проверить, действительно ли инструкция `LanguageOrgan.ts` держится
под давлением, а не только в спокойных условиях.

200. Проверить поведение при недоступности обоих backend'ов (Kobold и
Gemini одновременно) — деградирует ли организм гладко (честно
сообщает о невозможности говорить) или падает с необработанной
ошибкой.
201. Замерить реальное качество ответов при `preferredBackend='kobold'`
(локальная модель) против `'gemini'` — насколько сильно теряется
в качестве ради офлайн-автономности, конкретными примерами
бок-о-бок.
202. Проверить, передаётся ли `[NETWORK]` / `[NEUROMOD]` / `[MEMORY]`
контекст (виденный в `buildCognitivePrompt`) КОРРЕКТНО при разных
объёмах — не обрезается ли важная информация при большом
`worldFacts.length` без приоритизации значимости.
203. Реализовать явную проверку "верности канону" (раздел 22) именно на
уровне генерируемого языковым слоем текста — отдельный, лёгкий
классификатор/эвристика поверх выхода LLM, ловящий явные отклонения
от `KurisuCanonProfile`.
204. Провести сравнение стоимости (латентность + токены) `p40SelectModel`
решений — действительно ли выбор бэкенда

оптимален для конкретной
задачи, или это фиксированное правило без
реальной адаптации.

25. Социальное познание и отношения

**Статус: ✅ `SocialWorldModel.ts` (18 строк) — простой, но РЕАЛЬНО
адаптивный: confidence растёт/падает от настоящей обратной связи, не
захардкожен.
`LongTermSocialMind.ts` / `RelationshipSelf.ts` не проверял
построчно.**

Шаги:
205. Прочитать `LongTermSocialMind.ts` и `RelationshipSelf.ts` целиком — проверить, действительно ли они используют накопленную историю `SocialWorldModel.predictions`, или существуют параллельно без
реальной связи.
206. Заменить упрощённые regex-паттерны в `SocialWorldModel.observe()` (`/помоги|сделай|можешь/` и подобные) на что-то более
контекстно-чувствительное — регулярки ловят только явные ключевые
слова, пропуская большинство реальных

формулировок той же
потребности.
207. Добавить многоагентную поддержку по-настоящему — сейчас структура `agent:string` предполагает разных людей, но проверить: реально ли
тестировалось поведение с несколькими разными собеседниками, или
только с вами одним.
208. Реализовать явную модель "что я знаю о желаниях/целях именно этого
человека" отдельно от "какое у него сейчас настроение" — сейчас,
похоже, `SocialWorldModel` смешивает оба в одной структуре
`predictedNeed` / `predictedValence`.
209. Провести longitudinal-тест: одна и та же линия отношений на 50+
синтетических "разговоров", проверить, растёт ли `confidence`
предсказуемо и не деградирует ли внезапно без причины.
210. Связать социальную модель с мотивацией привязанности (раздел 9,
шаг 120) — рост доверия должен реально влиять на внутреннее
состояние организма, а не оставаться изолированной социальной
метрикой.

ЧАСТЬ V — СОЗНАНИЕ, ДОКАЗАТЕЛЬСТВА, ПРОДУКТ

26. Сознание и evidence battery

Статус: ⚠️ M1-M10 чек-лист, `ConsciousnessEvidenceLedger.ts`, `MetaCognitiveMonitor.ts`, `IntegrationPhiProxy` существуют. Consciousness evidence score = 8/100 по вашей же честной оценке (B265) — и это правильно честная низкая оценка, не то, что нужно "поднимать" искусственно.

Шаги:

211. НЕ пытаться поднять число 8/100 напрямую — это симптом честности, не проблема для решения. Вместо этого фокус — на качестве adversarial-тестирования гипотез (см. следующие шаги).

212. Реализовать полную adversarial-батарею: для каждой гипотезы сознания (GWT/recurrent processing/higher-order/predictive processing/IIT-inspired) — явный контр-эксперимент, пытающийся ОПРОВЕРГНУТЬ, а не подтвердить её применимость к организму.

213. Прочитать `MetaCognitiveMonitor.ts` (29 строк — очень маленький)
целиком, оценить, достаточно ли этого объёма кода для содержательного
метапознания, или это тоже кандидат на "измеряет, но не вычисляет"
паттерн, как `MicroDreamEngine`.
214. Явно связать `ConsciousnessEvidenceLedger` с `lesion()`-механизмом
`P31FunctionalConsciousness` — каждая запись в ledger должна
ссылаться на конкретный lesion/ablation эксперимент, который её
подтверждает, не быть голым утверждением.
215. Реализовать Attention Schema (Grazi
ano) компонент, предложенный в самом первом документе этой серии —
явную, инспектируемую модель "на что я сейчас, по своему
собственному представлению, обращаю внимание" — отдельно от
реального `GlobalWorkspaceSubstrateV2`-победителя, чтобы можно
было измерить РАСХОЖДЕНИЕ между тем, что система думает о своём
внимании, и тем, что реально происходит (это и есть содержательный
научный вопрос, а не просто ещё одна метрика).
216. Никогда не выставлять "VERIFIED" статус для утверждений о
сознании — только "hypothesis record + adversarial

battery",
как явно требует ваш `Function Passport` принцип.

27. Продукт: две ветки, UI, интеграция

**Статус: ❌ не решено ни в одном документе — у вас параллельно
существуют телефонное приложение
(`v96work` /v99.80, React/Vite/Live2D в
браузере) и эта исследовательская ветка (`35.6.1` ,
свой лёгкий UI,
гораздо более глубокое когнитивное ядро).**

Шаги:
217. Явно провести сравнение "что уникального есть только в v99.80"
(например: более полный self-code-lab HITL UI для нетехнического
пользователя, если он там более развит) против "что уникального
есть только в 35.6.1" (вся глубина B259-266, zero-dependency).
218. Принять явное решение: (a) 35.6.1 полностью заменяет v99.80 как
основной продукт, (b) когнитивное ядро `src/organism` переносится
внутрь v99.80's UI-обвязки, (c) обе ветки продолжают жить
раздельно намеренно (например, 35.6.1 —
"лаборатория", v99.80 —

"стабильный продукт", с явным процессом переноса проверенных
идей из первой во вторую).
219. Какой бы вариант ни был выбран — задокументировать его явно одним
файлом `BRANCH_STRATEGY_RU.md` в корне, чтобы это не терялось
между большими проходами.
220. Если выбран вариант с переносом — составить отдельный, специальный
план миграции (это отдельная большая работа, не влезающая в этот
документ) — не пытаться делать перенос попутно с продолжением
научной части.
221. Провести пользовательский тест (вы сами, как единственный
пользователь) — какая из двух веток сейчас реально приятнее и
полезнее использовать день за днём — это тоже данные для решения
из шага 218, не только архитектурные соображения.
222. Обновить `README.md` / `README_RU.md` так, чтобы явно указывать
статус выбранной стратегии — сейчас человек, открывший любой из
двух архивов, не может понять их отношение друг к другу без
внешнего контекста (как этот документ).

28. Что нужно для полной реализации проекта — сведено воедино

Финальный, самый прямой список из этого документа:

223. Кирпич 0 (autoRun) должен быть закрыт раньше, чем что-либо ещё —
без него "автономный организм" не является фактом.
224. Верификационный долг (65.8% UNVERIFIED на последнем известном
срезе) должен активно сокращаться каждый большой проход, не только
измеряться.
225. Первый настоящий прогон на физическом Android-устройстве — до
дальнейшего наращивания научной глубины, чтобы не строить месяцы
работы поверх непроверенного фундамента.
226. Явное решение по стратегии двух веток (раздел 27) — иначе усилия
продолжат распыляться.
227. Целенаправленное закрытие BUILD 4/11/12/13 (real local plasticity,
self-model, agency, motivation как learned dynamics) — отложенных
ради B262-266, но не отменённых.
228. Отдельный, целенаправленный проход по границе personality vs safety
(раздел 22, шаг 190) — явно откладываемый уже

третий документ
подряд; в какой-то момент это должно стать приоритетом, а не
вечно следующим пунктом.
229. Curiosity-Gated Action Diversification Buffer (или любой другой
метод, реально атакующий недостаток action/consequence diversity)
— потому что без этого причинная цепочка продолжит измерять одну
и ту же нерешённую проблему до бесконечности.
230. Честность как процесс (Function Passport, Neural Reality Gate,
Brain Lesion Matrix) должна применяться к КАЖДОМУ новому элементу
из частей III-IV этого документа по мере его проверки — не
оставаться методологией только для причинной цепочки B259-267.